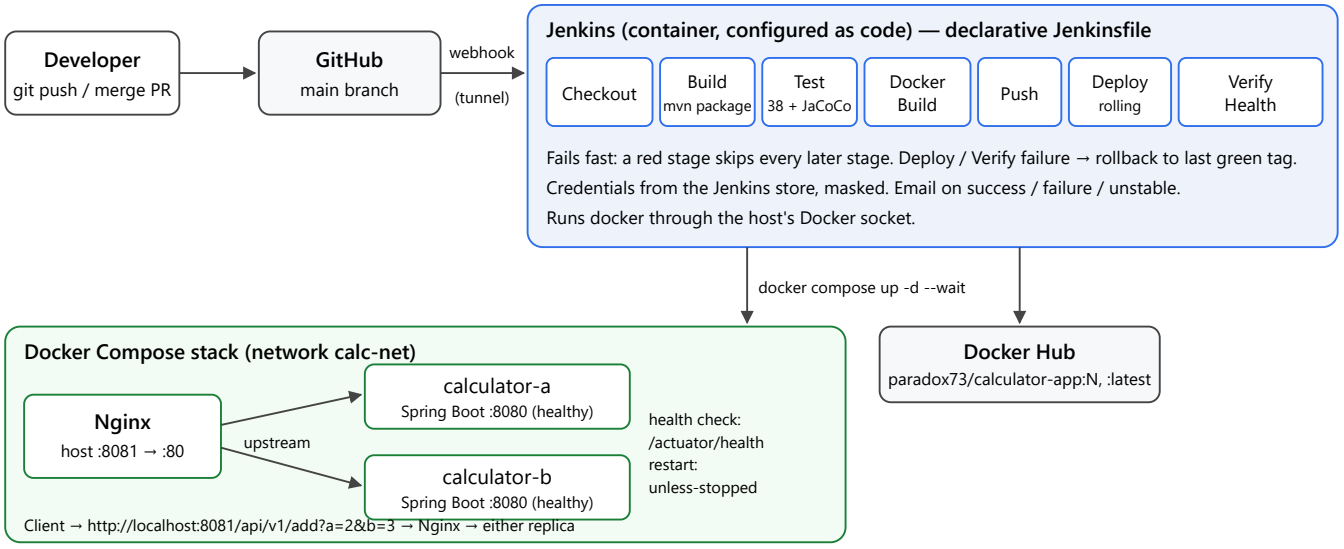
End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering, Mini-Project · Kanav Bhardwaj (IMT2023024) · Term I, 2026-27 · Faculty: Prof. B. Thangaraju

GitHub	github.com/Paradox-73/Calculator_REST_API
Docker Hub	hub.docker.com/r/paradox73/calculator-app (public; one tag per build number, plus `latest`)
Stack	Java 21, Spring Boot 3.5.16, Maven 3.9, JUnit 5 + MockMvc + JaCoCo, Docker, Docker Compose, Nginx 1.27, Jenkins LTS

1. Architecture of the pipeline



2. The application

A Spring Boot REST API with `GET /api/v1/{add,subtract,multiply,divide}?a=&b=` and Actuator's `/actuator/health`. Success returns `{"operation":"add","a":2.0,"b":3.0,"result":5.0}`. A missing or non-numeric parameter, `NaN` / `Infinity`, an overflowing result, and division by zero all return HTTP 400 with a structured body (timestamp, status, error, message, path) from one `@RestControllerAdvice`. Arithmetic lives in `CalculatorService`, apart from the HTTP layer.

Tests (38, all passing): 29 plain JUnit 5 unit tests of the service (5 cases per operation, divide by zero and negative zero, non-finite inputs, overflow); 7 MockMvc tests of the web layer (every operation and every error path); 2 tests on a real embedded server (health is `UP`, `add` returns 5.0). Surefire writes JUnit XML for Jenkins. JaCoCo measures 90% instruction and 100% branch coverage and fails the build below 85%.

3. Pipeline stages

Stage	What it does	Fails when
Trigger	GitHub webhook (`githubPush()`) through a Cloudflare tunnel to the local Jenkins; `pollSCM('H/5 * * * *')` as a fallback. A merged PR starts a build within about 10 s.	—
Checkout	`checkout scm`; keeps the short commit hash for the email.	GitHub unreachable
Build	`mvn -B clean package -DskipTests`; archives the JAR.	Compile error
Test	`mvn -B test`; `junit` publishes `target/surefire-reports/*.xml` (`allowEmptyResults: false`); JaCoCo report archived.	A failing test, zero test reports, coverage < 85%
Docker Build	`docker build` of the multi-stage Dockerfile, tagged `:BUILD_NUMBER` and `:latest`; prints the image size.	Dockerfile error

Push	`withCredentials` reads `dockerhub-credentials` from the Jenkins store; `docker login --password-stdin`; pushes both tags; `docker logout` in `post`.	Bad token, network
Deploy	Rolling update: `docker compose up -d --wait calculator-a`, 10 s pause, same for `calculator-b`, then Nginx. One replica always serves traffic.	Container not healthy
Verify Health	Polls `/actuator/health` through Nginx (12 × 5 s), then checks that `add?a=2&b=3` returns `"result":5.0`.	Stack not healthy or wrong answer
post	Success / failure / unstable emails (failure attaches the log and names the stage); `docker image prune`. A Deploy or Verify failure redeploys the last green build.	—

Re-runnable and idempotent: `disableConcurrentBuilds()` allows one deploy at a time; `compose up` only replaces containers whose image changed. Measured: during a redeploy (build 5) 146 of 146 requests to the API succeeded. A deliberately broken replay (build 6) deployed tag 6, failed the smoke test, and was rolled back to tag 5 automatically.

Containerization

- **Dockerfile:** stage 1 `maven:3.9-eclipse-temurin-21` copies `pom.xml` and runs `dependency:go-offline` before copying `src` (dependency layer cached); stage 2 `eclipse-temurin:21-jre-alpine` copies only the JAR, runs as non-root user `spring`, `EXPOSE 8080`, `HEALTHCHECK`, `exec java` so the JVM gets SIGTERM, heap sized to the container (`MaxRAMPercentage=75`). Base images pinned by `sha256` digest. `.dockerignore` keeps `target/`, `.git/`, docs and `.env` out. Image: 95 MB (`docker image inspect`). Runs alone with `docker run -p 9090:8080 paradox73/calculator-app`.
- **docker-compose.yml:** network `calc-net`; two API services from one YAML anchor (health check on `/actuator/health`, `restart: unless-stopped`, environment variables, 512 MB limit); Nginx reverse proxy as the only host port (8081), started only when both replicas are healthy. One `docker compose up` starts the full stack.

Version control

Progressive commits on `main` (skeleton → service → tests → controller → Docker → Compose → Jenkinsfile → README), then four feature/fix branches merged through pull requests #1–#4 with merge commits. `.gitignore` excludes `target/`, IDE files and every secret file; Jenkins secrets live only in the gitignored `jenkins/.env`.

4. Challenges faced

Problem	Cause and fix
Redeploy dropped requests (8 of 48 failed)	`compose up` recreated both replicas at once. Fix: two services and a rolling update. A first attempt still failed about 40% of requests: the new replica was healthy in 4 s, before Nginx had re-resolved its address, so both were unreachable together. Fix: 1 s DNS refresh, 1 s connect timeout, 10 s pause between replicas. Result: 0 failures.
Nginx health check never passed	`localhost` in Alpine resolves to IPv6 `::1`; Nginx listens on IPv4 only. Fix: probe `127.0.0.1`.
Push reached Jenkins but no build started	Configuration as Code re-creates the job on every Jenkins start, wiping the triggers the first run registers. Fix: declare `githubPush()` and `pollSCM` in the job definition too (PR #3).
Jenkins in a container	Bind-mounted `nginx.conf` resolves on the host, not in the Jenkins workspace → config baked into an Nginx image. `localhost` in Jenkins is the Jenkins container → Verify Health uses `host.docker.internal`.
Webhook needs a public URL	Jenkins runs on a laptop. A Cloudflare quick tunnel gives it an HTTPS address; `jenkins/update-webhook.sh` repoints the GitHub webhook after each restart. Polling remains as fallback.
Failure email crashed	`attachLog` in email-ext needs the `matrix-project` plugin; added to `plugins.txt`.

Appendix: screenshots

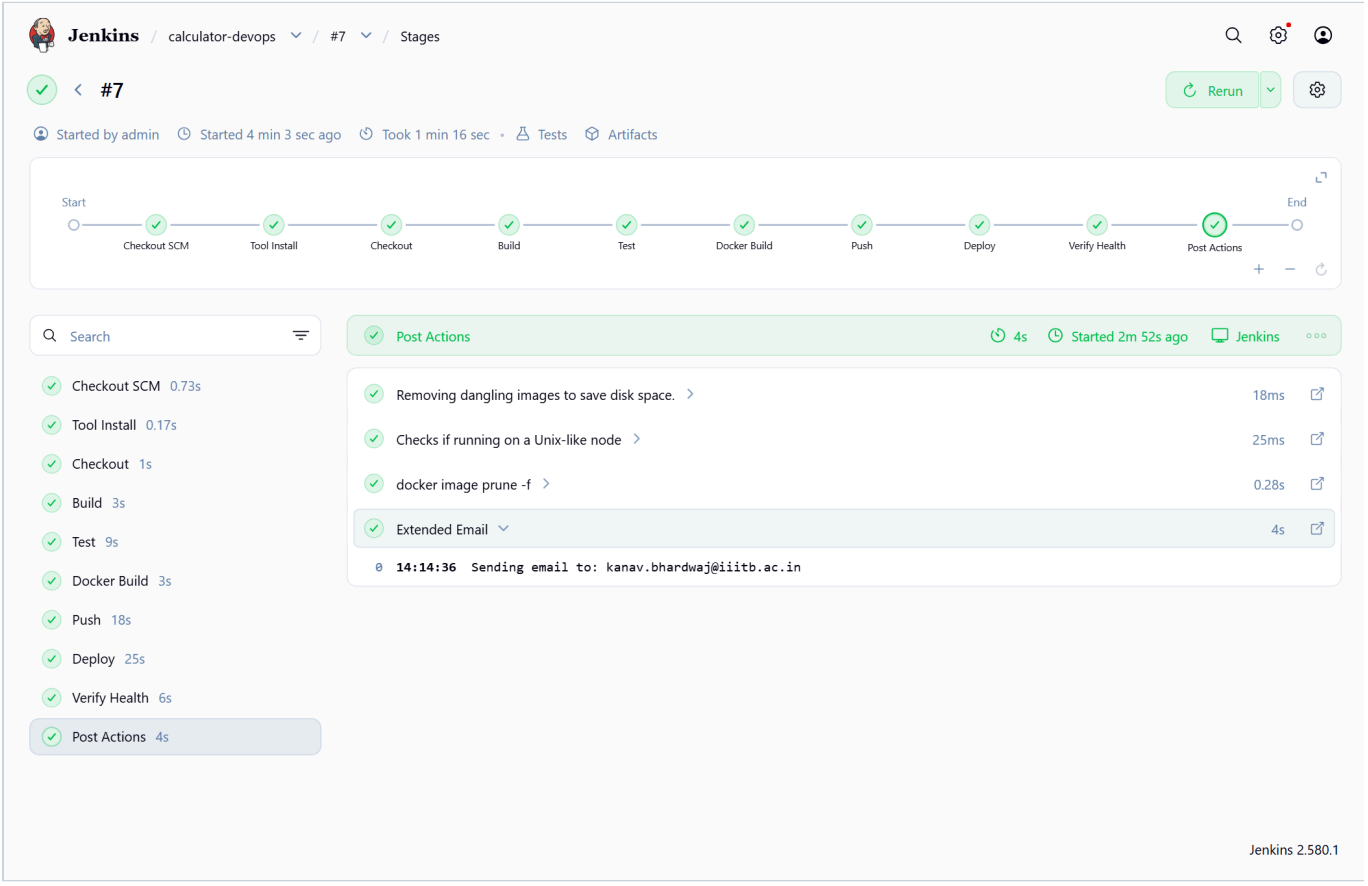


Figure 1. Green Jenkins run #7: all stages passed, success email sent (Post Actions).

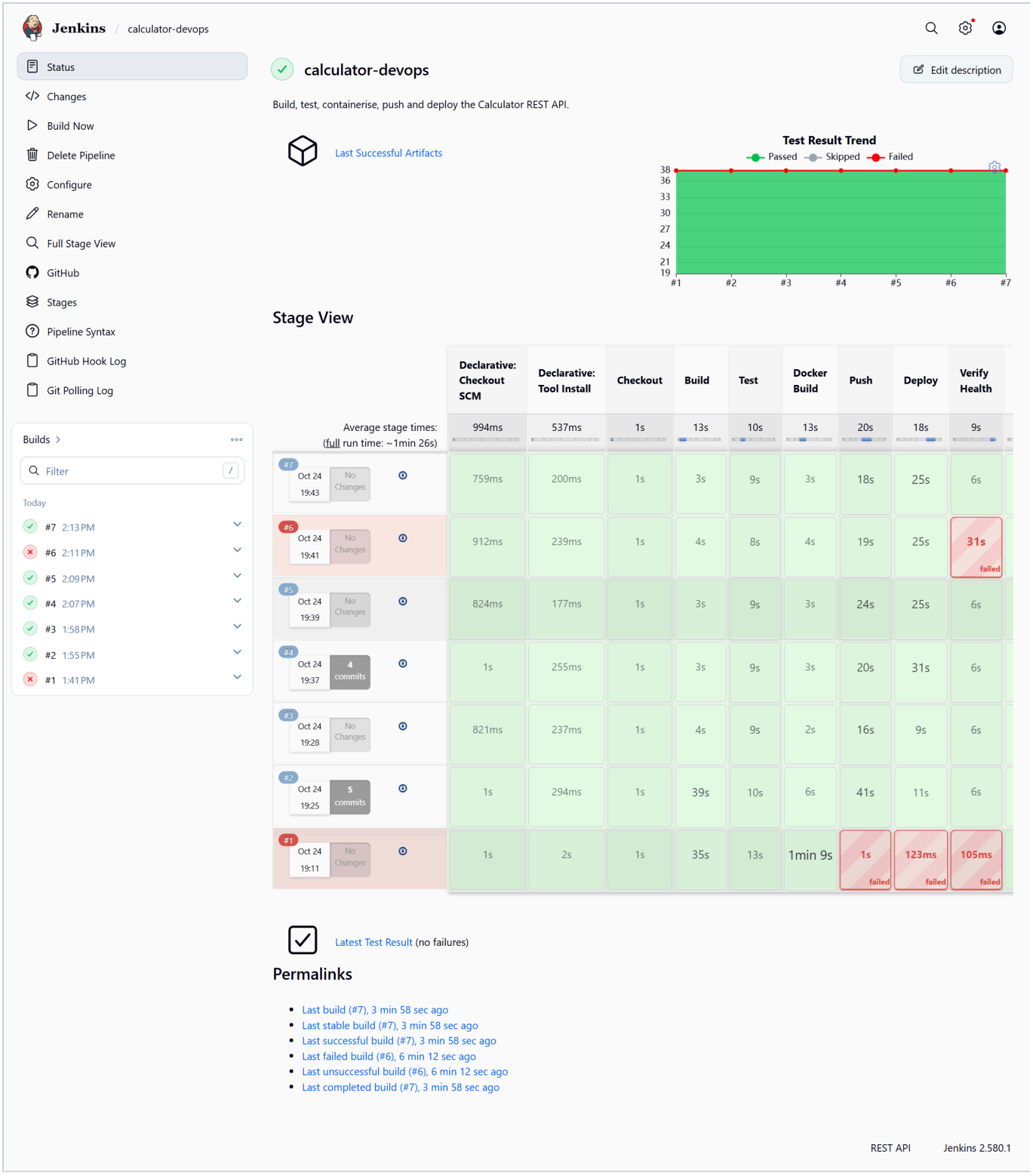


Figure 2. Stage view of builds #1-#7. #1 failed at Push before the Docker Hub token was set; #6 is the deliberate smoke-test failure that rolled back to #5.

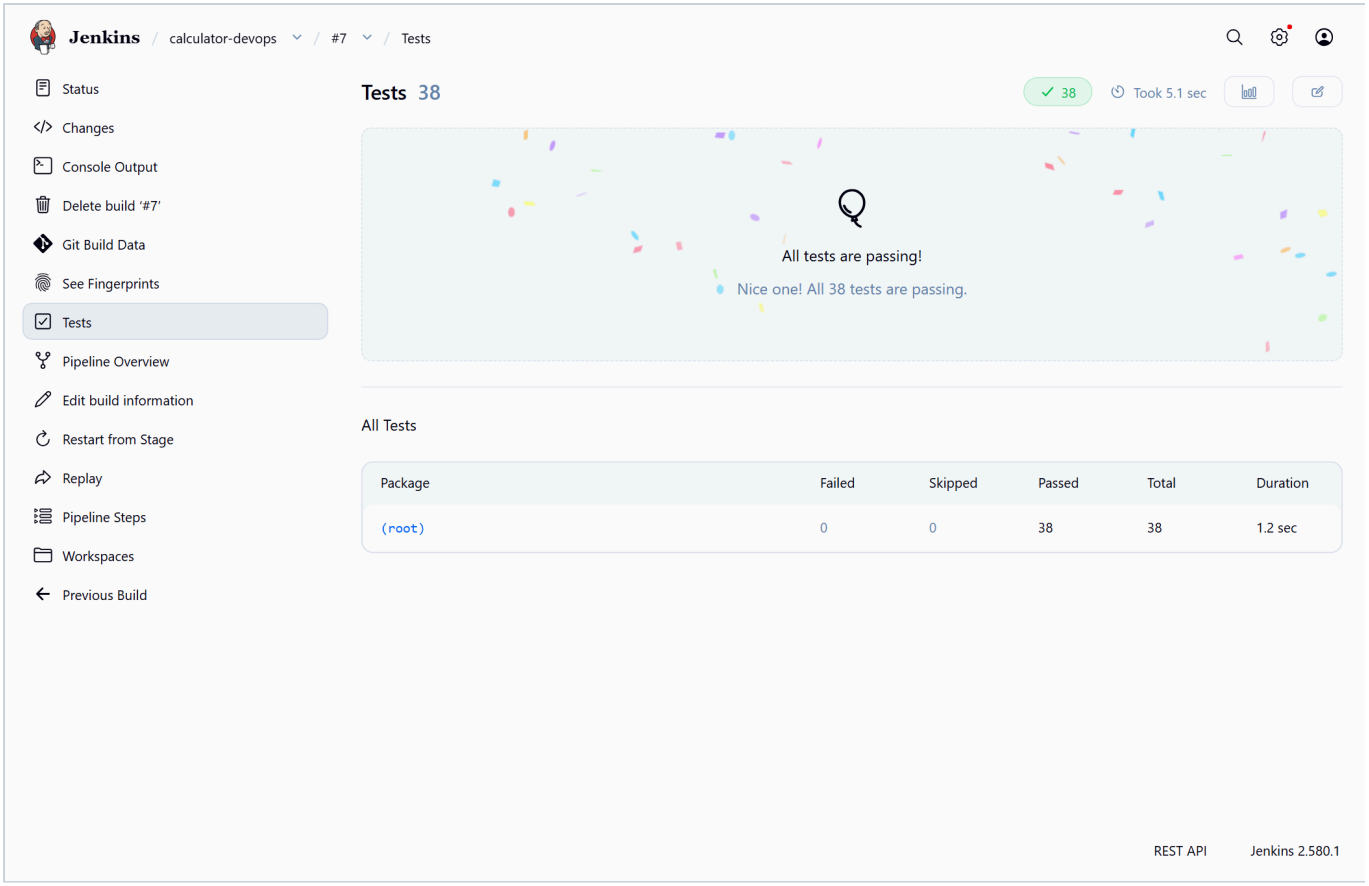


Figure 3. JUnit results published by Jenkins: 38 tests, 0 failures.

```
Terminal output: docker ps (Jenkins build #7 deployed)

$ docker ps
NAMES                              IMAGE                              STATUS                   PORTS
calculator-devops-calculator-b-1   paradox73/calculator-app:7         Up 2 minutes (healthy)   8080/tcp
calculator-devops-calculator-a-1   paradox73/calculator-app:7         Up 3 minutes (healthy)   8080/tcp
calculator-devops-nginx-1          edcd8836cf36                       Up 8 minutes (healthy)   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
jenkins-jenkins-1                  calculator-jenkins:latest          Up 22 minutes            0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
jenkins-tunnel-1                   cloudflare/cloudflared:2025.10.0   Up 25 minutes
```

Figure 4. docker ps: two healthy API replicas from image tag 7 and the Nginx proxy on port 8081.

```
Terminal output: sample API calls through Nginx on port 8081

$ curl "http://localhost:8081/api/v1/add?a=2&b=3"
{"operation":"add","a":2.0,"b":3.0,"result":5.0}  [HTTP 200]

$ curl "http://localhost:8081/api/v1/subtract?a=10&b=4"
{"operation":"subtract","a":10.0,"b":4.0,"result":6.0}  [HTTP 200]

$ curl "http://localhost:8081/api/v1/multiply?a=4&b=2.5"
{"operation":"multiply","a":4.0,"b":2.5,"result":10.0}  [HTTP 200]

$ curl "http://localhost:8081/api/v1/divide?a=7&b=2"
{"operation":"divide","a":7.0,"b":2.0,"result":3.5}  [HTTP 200]

$ curl "http://localhost:8081/api/v1/divide?a=1&b=0"
{"timestamp":"2026-10-24T14:17:19.7730732242","status":400,"error":"Division by zero","message":"Division by zero is not allowed. Give a non-zero value for
'b'.","path":"/api/v1/divide"}  [HTTP 400]

$ curl "http://localhost:8081/api/v1/add?a=2"
{"timestamp":"2026-10-24T14:17:19.900569356Z","status":400,"error":"Missing parameter","message":"Required parameter 'b' is missing.","path":"/api/v1/add"}  [HTTP 400]

$ curl "http://localhost:8081/api/v1/add?a=abc&b=1"
{"timestamp":"2026-10-24T14:17:20.022849170Z","status":400,"error":"Invalid parameter","message":"Parameter 'a' must be a number. Received:
'abc'.","path":"/api/v1/add"}  [HTTP 400]

$ curl "http://localhost:8081/actuator/health"
{"status":"UP","groups":["liveness","readiness"]}  [HTTP 200]
```

Figure 5. Sample API responses through the proxy, including the HTTP 400 error bodies.

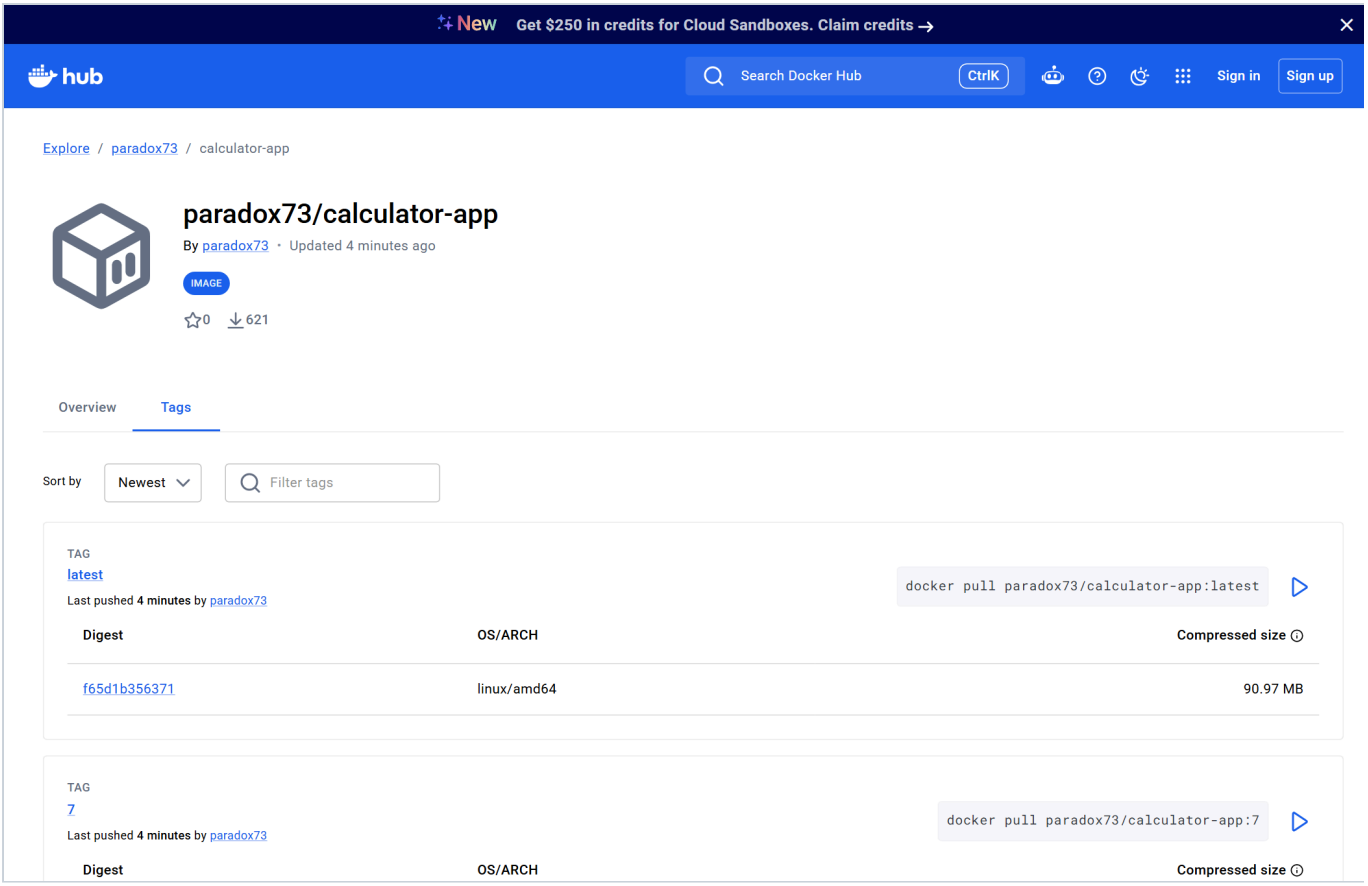


Figure 6. Public Docker Hub repository paradox73/calculator-app with build-number tags.

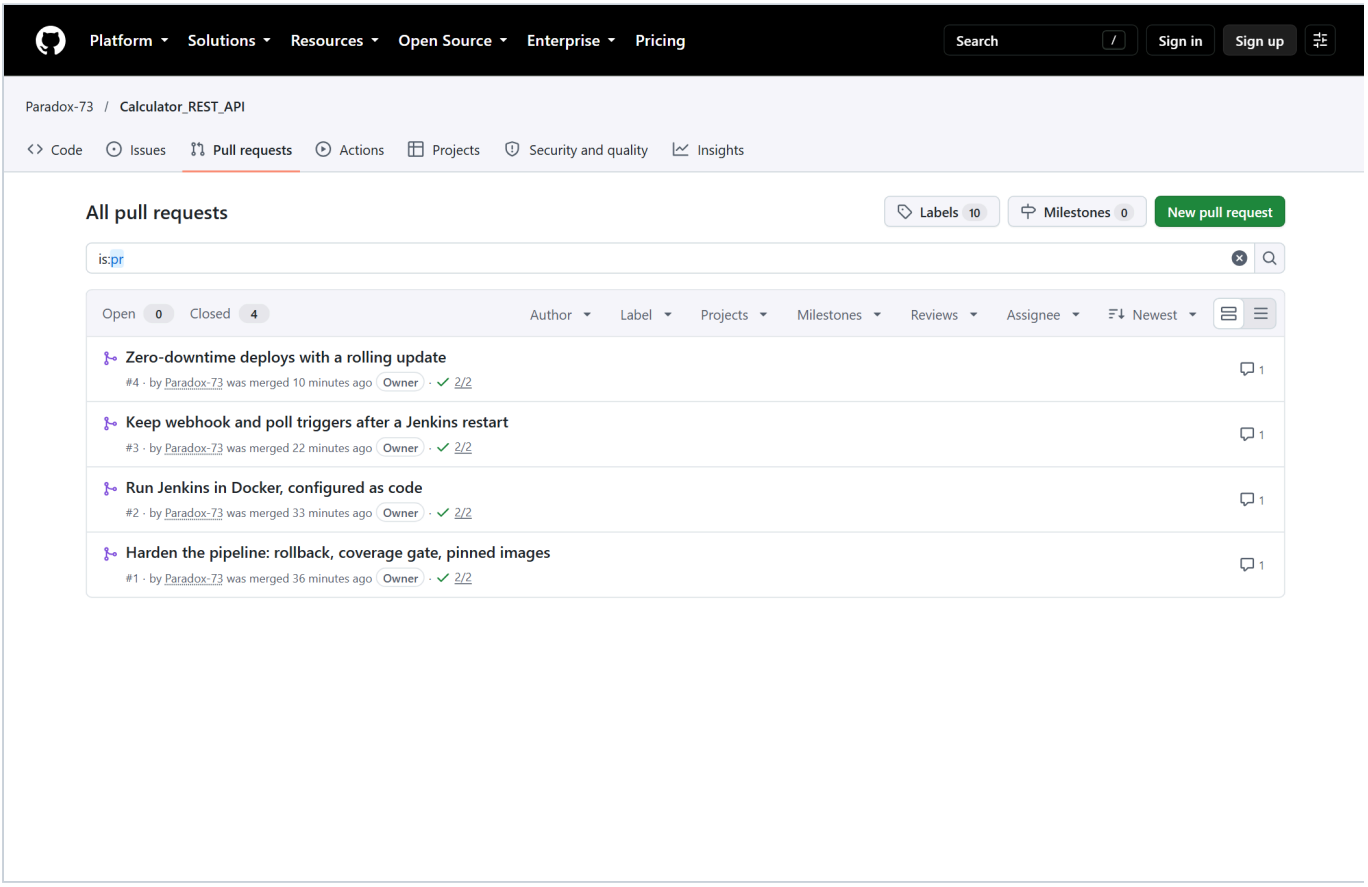


Figure 7. GitHub: four feature/fix branches merged through pull requests.

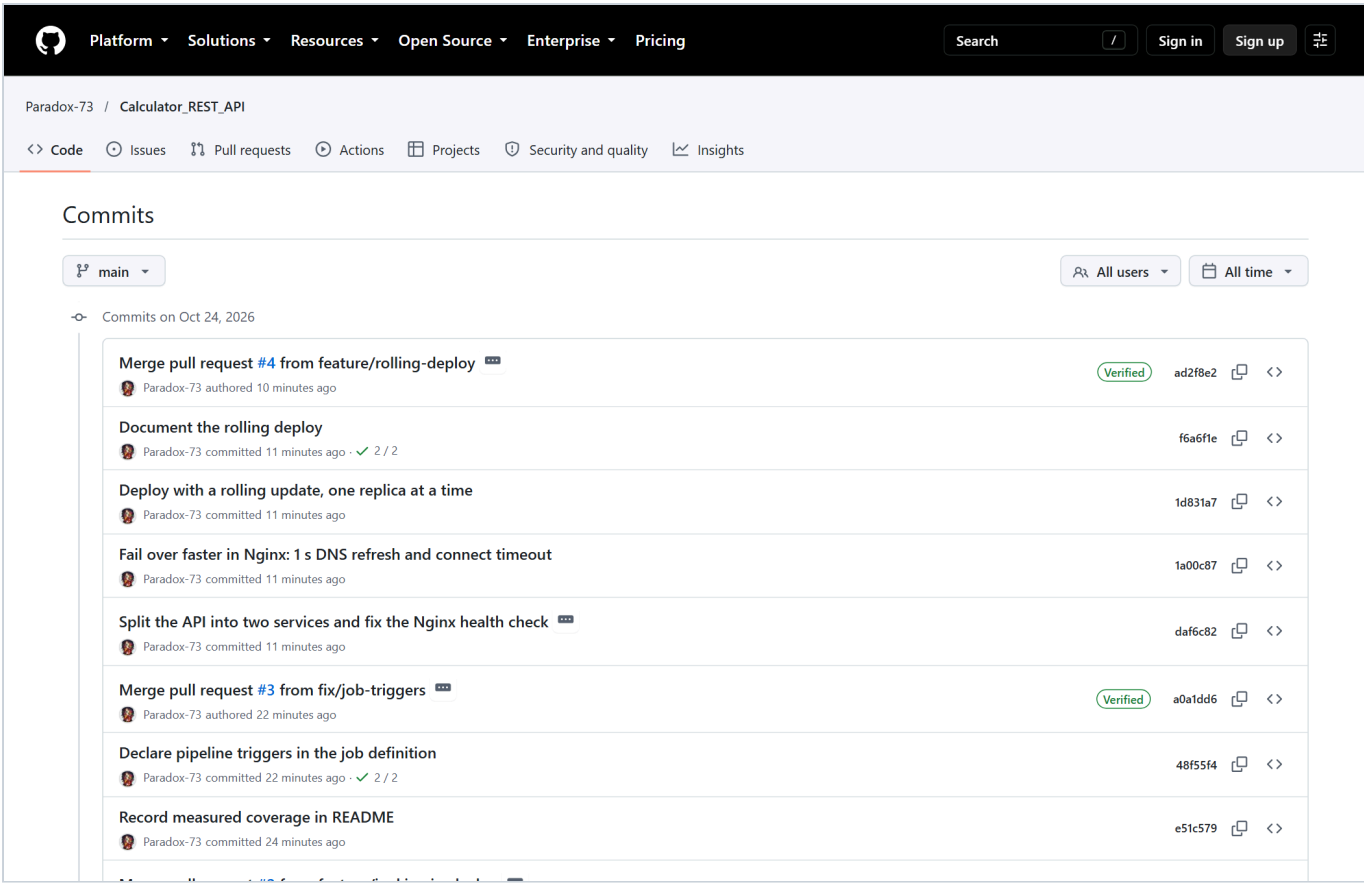


Figure 8. GitHub: progressive commit history on main.